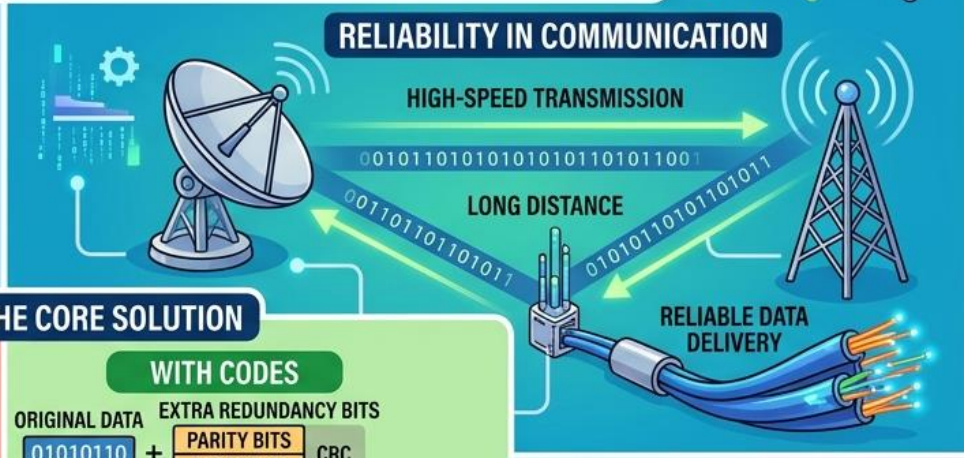
CODIGOS DE CORRECION Y DETECCION DE ERRORES

Los códigos de detección y corrección de errores surgieron por una necesidad fundamental: el **ruido**. En cualquier sistema de comunicación o almacenamiento, los datos viajan a través de medios imperfectos (cables, ondas de radio, discos magnéticos) donde factores como la interferencia electromagnética o el calor pueden alterar los bits (cambiar un 0 por un 1).

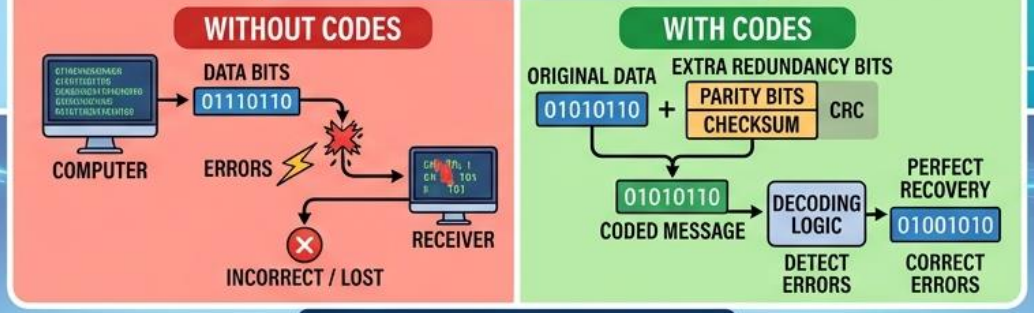
WHY WERE ERROR DETECTION & CORRECTION CODES CREATED?

THE PROBLEM: DIGITAL DATA ISSUES

RELIABILITY IN COMMUNICATION



HOW IT WORKS: THE CORE SOLUTION



KEY BENEFITS & APPLICATIONS

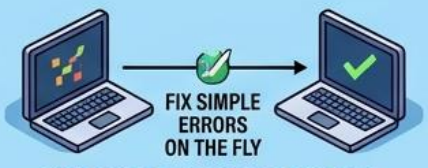
INCREASED DATA INTEGRITY

PREVENT DATA CORRUPTION

EFFICIENT RE-TRANSMISSION

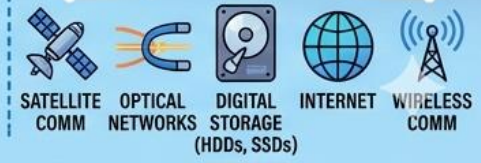
DETECT AND ASK TO RE-SEND

FORWARD ERROR CORRECTION (FEC)



FIX SIMPLE ERRORS ON THE FLY

ENABLES CRITICAL SYSTEMS



CRONOLOGÍA DE CÓDIGOS DE DETECCIÓN Y CORRECCIÓN DE ERRORES (ECC): DESARROLLO HISTÓRICO Y APLICACIONES ACTUALES

1940s-1950s

CÓDIGO DE HAMMING (1950)
(e.g., [7,4])
Richard Hamming
Códigos de corrección de un solo error.

APLICACIONES ACTUALES
- Memoria de computadora (DRAM con ECC)
- Redes de área local (e.g., Ethernet, CRC simple)
- Señalización básica

CÓDIGO REED-MULLER (1954)
Irving S. Reed y David E. Muller

Códigos lineales y de potencia.

APLICACIONES ACTUALES
- Memoria de computadora (DRAM con ECC)
- Redes de área local (e.g., Ethernet, CRC simple)
- Señalización básica

Códigos lineales y de potencia.

APLICACIONES ACTUALES
- Transmisiones satelitales tempranas (p.ej., Mariner 9)
- Redes inalámbricas de sensores (WSN)
- Comunicaciones profundas del espacio (en combinación)

1960s

CÓDIGO REED-SOLOMON (1960)
(RS)
Irving Reed y Gustave Solomon
Códigos de corrección de ráfagas, basados en campos de Galois.

APLICACIONES ACTUALES
- Almacenamiento óptico (CDs, DVDs, Blu-ray)
- Códigos QR
- Transmisiones de televisión digital (DVB, ATSC)
- Estándares de comunicación (e.g., DOCSIS)

CÓDIGOS BÁSE-CHAUDHURI-HOCQUENGHEM (BCH) (1959-1960)
Alexis Hocquenghem, Raj Bose, D. K. Ray-Chaudhuri
Códigos cíclicos para errores de bits múltiples.

APLICACIONES ACTUALES
- Discos duros modernos
- Unidades de estado sólido (SSDs)
- Sistemas de comunicación satelital

1970s-1980s

CÓDIGOS CONVOLUCIONALES (Años 60-70)

Códigos que procesan datos de forma continua.

APLICACIONES ACTUALES
- Comunicaciones móviles (GSM, 3G, 4G LTE)
- Satélites de comunicación
- Deep Space Network (p.ej., NASA Voyager)

1990s

CÓDIGOS TURBO (1993)
Claude Berrou, Alain Glavieux, Punya Thitimajshima

Unen códigos convolucionales con iteraciones

APLICACIONES ACTUALES
- 4G LTE
- Algunos sistemas de satélite de alto rendimiento (GEO)
- Estándares de comunicación específicos

2000s-PRESENTE

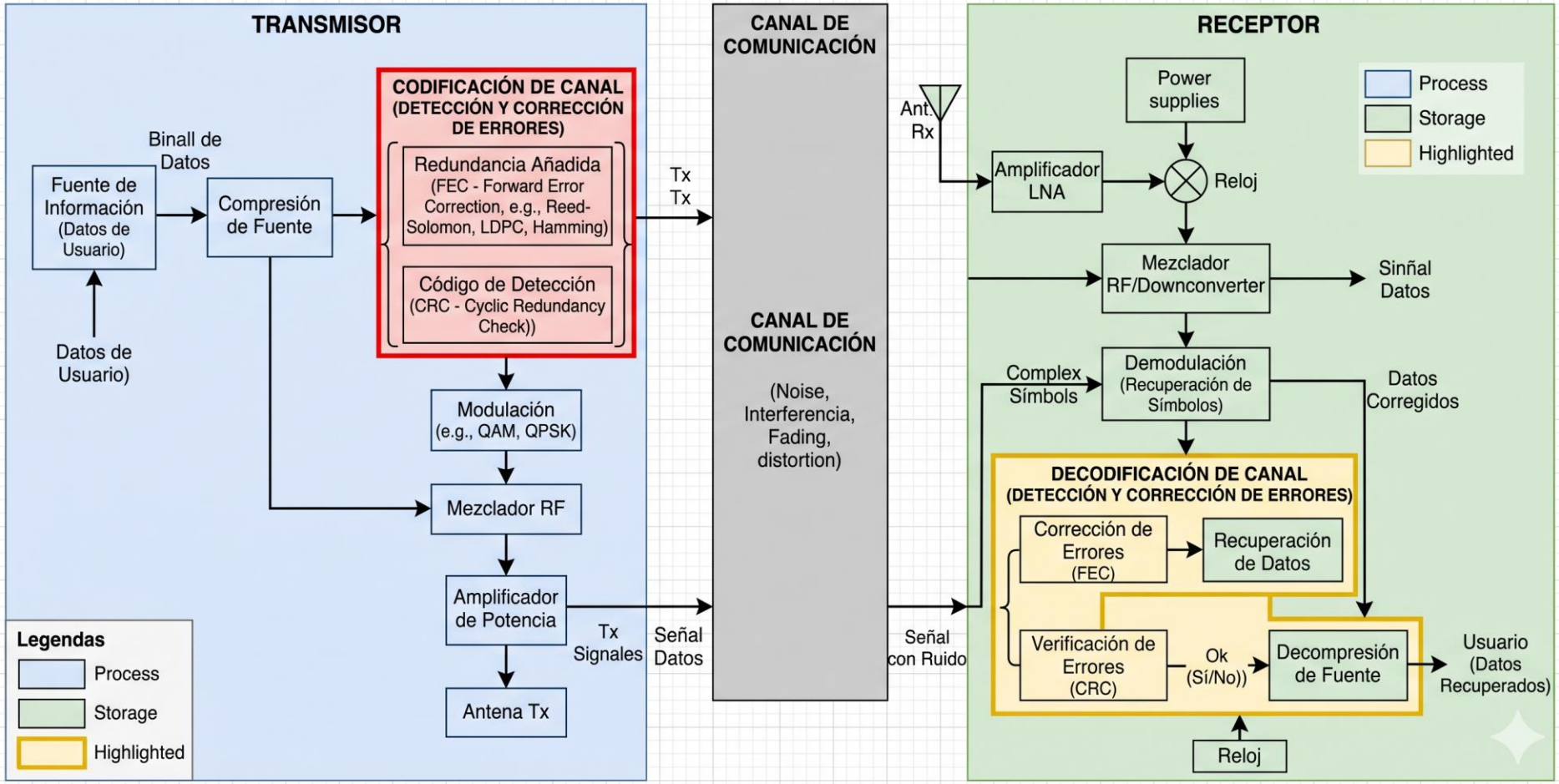
CÓDIGOS DE POLAR (2008)
Erdal Arıkan
Basados en la polarización del canal

APLICACIONES ACTUALES
- Quinta generación de comunicaciones móviles (5G) (especialmente canales de control)
- Redes ópticas de ultra alta velocidad

CÓDIGOS LDPC (REDESCUBIERTOS)
(Originado en 1962 por Gallager, redescubierto finales de los 90)
Códigos con matrices de paridad dispersas

APLICACIONES ACTUALES
- 5G (canales de datos)
- Televisión digital por satélite y cable modernas (e.g., DVB-S2, ATSC 3.0)
- Wi-Fi moderno (802.11n/ac/ax)
- Discos duros de muy alta densidad

ESQUEMA COMPLETO DE TRANSMISIÓN Y RECEPCIÓN CON DETECCIÓN Y CORRECCIÓN DE ERRORES



1 0 1 1 0 0 1 0 1 1 0 1 0 0 1

CÓDIGO DE HAMMING

Detección y Corrección de Errores en Transmisión Digital

Comunicaciones Digitales & Redes | NET3MAX – Cusco, Perú

Richard W. Hamming | Bell Labs, 1950

0 1 0 1 1 0 0 1 1 0 1 0 1 1 0

Contexto Histórico

Richard Wesley Hamming

1915
-
1998

Matemático & Informático
Bell Telephone Laboratories
New Mexico Tech, US Naval PG School

1947

Hamming trabaja en las computadoras de relays en Bell Labs. Los errores de cinta perforadora causan reinicios de fin de semana.

1950

Publica 'Error Detecting and Error Correcting Codes' en Bell System Technical Journal.

1960s

El código se adopta en computadoras mainframe IBM para protección de memoria (ECC RAM).

Hoy

Omnipresente en memorias ECC, comunicaciones espaciales, almacenamiento RAID, DVDs y redes.

Fuente: Hamming, R.W. (1950). Bell System Technical Journal, 29(2), 147–160.

Errores en Transmisión Digital

Error de bit único

1 bit

Un solo bit cambia de 0→1 o 1→0.
Es el tipo más común en memoria RAM y canales con bajo ruido.
El código Hamming lo detecta Y corrige.

Error de doble bit

2 bits

Dos bits alterados simultáneamente.
Hamming(SECDED) puede detectarlo pero NO corregirlo automáticamente.
Requiere retransmisión.

Error en ráfaga (burst)

N bits

Múltiples bits consecutivos corrompidos.
Causados por interferencia eléctrica, fading en RF.
Requiere códigos más potentes: Reed-Solomon, LDPC.

Concepto clave — distancia de Hamming: número de bits que difieren entre dos palabras código.

Principios Teóricos del Código de Hamming

Condición fundamental: $2^r \geq m + r + 1$ donde r = bits de paridad, m = bits de datos

01

Posiciones de paridad

Los bits de paridad ocupan las posiciones que son potencias de 2: 1, 2, 4, 8, 16...
Cada uno cubre un subconjunto de bits de datos.

02

Paridad par (even)

El XOR de todos los bits en cada grupo cubierto debe ser 0.
Si hay un error, uno o más bits de paridad fallarán.

03

Síndrome de error

Los bits de paridad fallados forman un número binario (síndrome) que indica la posición exacta del bit erróneo.

04

SECDED

Single Error Correction / Double Error Detection.
Un bit extra (p_total) permite detectar 2 errores simultáneos.

Lin, S. & Costello, D.J. (2004). Error Control Coding. Prentice Hall.

La idea consiste en agregar **bits de paridad** a un conjunto de datos. Estos bits de paridad se colocan en posiciones especiales:

- posición 1
- posición 2
- posición 4
- posición 8
- etc.

Es decir, en las posiciones que son potencias de 2.

La cantidad de bits de paridad `r` debe cumplir:

$$2^r \geq m + r + 1$$

Donde:

- `m` = número de bits de información
- `r` = número de bits de paridad

EJEMPLO 1: Nivel básico – Hamming (7,4)

Aquí se usan:

- **4 bits de datos**
- **3 bits de paridad**
- total: **7 bits**

Paso 1: Datos a transmitir

Supongamos que queremos transmitir:

$$1011$$

Llamemos a los bits de datos:

- D1 = 1
- D2 = 0
- D3 = 1
- D4 = 1

Paso 2: Determinar cuántos bits de paridad se necesitan

Tenemos `m = 4`.

Probamos:

- si `r = 2`:

$$2^2 = 4 \ngeq 4 + 2 + 1 = 7$$

No alcanza.

- si `r = 3`:

$$2^3 = 8 \geq 4 + 3 + 1 = 8$$

Sí alcanza.

Entonces necesitamos 3 bits de paridad:

- P1
- P2
- P4

Paso 3: Ubicar los bits en sus posiciones

Las posiciones del código de 7 bits serán:

Posición	Tipo
1	P1
2	P2
3	D1
4	P4
5	D2
6	D3
7	D4

Sustituyendo:

Posición	Valor
1	P1
2	P2
3	1
4	P4
5	0
6	1
7	1

Paso 4: Calcular las paridades

Usaremos **paridad par**, es decir, cada grupo debe tener un número total par de unos.

Paridad P1

P1 revisa las posiciones cuyos números en binario tienen el bit menos significativo en 1:

- 1
- 3
- 5
- 7

Entonces revisa:

- posición 1 = P1
- posición 3 = 1
- posición 5 = 0
- posición 7 = 1

Sin contar aún P1, los valores conocidos son:

$$1 + 0 + 1 = 2$$

Ya hay cantidad par de unos, así que:

$$P1 = 0$$

Paridad P2

P2 revisa posiciones:

- 2
- 3
- 6
- 7

Valores:

- posición 2 = P2
- posición 3 = 1
- posición 6 = 1
- posición 7 = 1

Sin contar P2:

$$1 + 1 + 1 = 3$$

Hay cantidad impar, entonces para que quede par:

$$P2 = 1$$

Paridad P4

P4 revisa posiciones:

- 4
- 5
- 6
- 7

Valores:

- posición 4 = P4
- posición 5 = 0
- posición 6 = 1
- posición 7 = 1

Sin contar P4:

$$0 + 1 + 1 = 2$$

Ya es par, entonces:

$$P4 = 0$$

Paso 5: Código final transmitido

Entonces el código completo es:

Posición	1	2	3	4	5	6	7
Valor	0	1	1	0	0	1	1

Código transmitido:

$$0110011$$

EJEMPLO 2: Detección y corrección de un error en Hamming (7,4)

Ahora supongamos que durante la transmisión se altera un bit y el receptor recibe:

$$0100011$$

Observa que el bit en la posición 3 cambió de `1` a `0`.

Paso 1: Verificar paridades en el receptor

Verificación de P1

P1 revisa posiciones 1, 3, 5, 7:

- 1 = 0
- 3 = 0
- 5 = 0
- 7 = 1

Suma de unos:

$$0 + 0 + 0 + 1 = 1$$

Resultado impar.
Entonces hay error en el grupo de P1.

$$S1 = 1$$

Verificación de P2

P2 revisa posiciones 2, 3, 6, 7:

- 2 = 1
- 3 = 0
- 6 = 1
- 7 = 1

Suma:

$$1 + 0 + 1 + 1 = 3$$

Impar, entonces:

$$S2 = 1$$

Verificación de P4

P4 revisa posiciones 4, 5, 6, 7:

- 4 = 0
- 5 = 0
- 6 = 1
- 7 = 1

Suma:

$$0 + 0 + 1 + 1 = 2$$

Par, entonces:

$$S4 = 0$$

Paso 2: Formar el síndrome

Se forma con el orden:

$$S4S2S1$$

Entonces:

$$011$$

En decimal:

$$3$$

Eso significa que el error está en la **posición 3**.

Paso 3: Corregir el bit

Bit recibido en posición 3 = 0

Se invierte:

$$0 \to 1$$

Código corregido:

$$0110011$$

Ahora extraemos los bits de datos de las posiciones:

- 3
- 5
- 6
- 7

Obtenemos:

$$1011$$

Se recuperó el dato original.

EJEMPLO 3: Nivel intermedio – Hamming con 8 bits de datos

Ahora trabajemos con un caso más práctico, más cercano a una implementación real.

Queremos transmitir:

$$11010110$$

Aquí:

- `m = 8`

Buscamos `r` tal que:

$$2^r \geq m + r + 1$$

Probamos:

- `r = 3` → $8 \ngeq 8 + 3 + 1 = 12$
- `r = 4` → $16 \geq 8 + 4 + 1 = 13$

Entonces se necesitan **4 bits de paridad**.

Total de bits:

$$8 + 4 = 12$$

Es decir, un **Hamming (12,8)**.

Paso 1: Ubicar posiciones

Las posiciones 1, 2, 4 y 8 serán paridades.

Las demás serán datos:

Posición	Tipo
1	P1
2	P2
3	D1
4	P4
5	D2
6	D3
7	D4
8	P8
9	D5
10	D6
11	D7
12	D8

Datos: `11010110`

Entonces:

- D1 = 1
- D2 = 1
- D3 = 0
- D4 = 1
- D5 = 0
- D6 = 1
- D7 = 1
- D8 = 0

Tabla inicial:

Posición	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Valor	P1	P2	1	P4	1	0	1	P8	0	1	1	0

Paso 2: Calcular paridades

P1 revisa:

1, 3, 5, 7, 9, 11

Valores conocidos:

- 3 = 1
- 5 = 1
- 7 = 1
- 9 = 0
- 11 = 1

Total de unos:

$$1 + 1 + 1 + 0 + 1 = 4$$

Ya es par, por tanto:

$$P1 = 0$$

P2 revisa:

2, 3, 6, 7, 10, 11

Valores conocidos:

- 3 = 1
- 6 = 0
- 7 = 1
- 10 = 1
- 11 = 1

Total:

$$1 + 0 + 1 + 1 + 1 = 4$$

Par, entonces:

$$P2 = 0$$

P4 revisa:

4, 5, 6, 7, 12

Valores conocidos:

- 5 = 1
- 6 = 0
- 7 = 1
- 12 = 0

Total:

$$1 + 0 + 1 + 0 = 2$$

Par, entonces:

$$P4 = 0$$

P8 revisa:

8, 9, 10, 11, 12

Valores conocidos:

- 9 = 0
- 10 = 1
- 11 = 1
- 12 = 0

Total:

$$0 + 1 + 1 + 0 = 2$$

Par, entonces:

$$P8 = 0$$

Paso 3: Código transmitido

El código queda:

$$001010100110$$

Paso 4: Supón un error

Imagina que durante la transmisión se altera la posición 10.

Código recibido:

$$001010100010$$

El receptor recalcula el síndrome.
Si el síndrome da:

$$1010_2 = 10$$

entonces el error está en la **posición 10** y se corrige invirtiendo ese bit.

Estructura de la Palabra Código — Hamming(7,4)

Bit de Paridad	Posiciones que cubre	Verificación XOR
p_1 (pos 1)	1, 3, 5, 7	$p_1 \oplus d_1 \oplus d_2 \oplus d_4 = 0$
p_2 (pos 2)	2, 3, 6, 7	$p_2 \oplus d_1 \oplus d_3 \oplus d_4 = 0$
p_3 (pos 4)	4, 5, 6, 7	$p_3 \oplus d_2 \oplus d_3 \oplus d_4 = 0$

Formato Hamming(7,4): 4 bits de datos + 3 bits de paridad = 7 bits totales | Detecta 2 errores, corrige 1.

Ejemplo Práctico — Codificación Hamming(7,4)

Dato a transmitir: $d_1d_2d_3d_4 = 1011$ → Calcular la palabra código Hamming(7,4)

1 **Ubicar datos en posiciones no-potencia de 2:**

Pos: $1(p_1)$ $2(p_2)$ $3(d_1{=}1)$ $4(p_3)$ $5(d_2{=}0)$ $6(d_3{=}1)$ $7(d_4{=}1)$

2 **Calcular bits de paridad (paridad par = XOR = 0):**

$p_1 = pos(1,3,5,7) = p_1 \oplus 1 \oplus 0 \oplus 1 = 0 \rightarrow p_1 = 0$

$p_2 = pos(2,3,6,7) = p_2 \oplus 1 \oplus 1 \oplus 1 = 0 \rightarrow p_2 = 1$

$p_3 = pos(4,5,6,7) = p_3 \oplus 0 \oplus 1 \oplus 1 = 0 \rightarrow p_3 = 0$

✔ Palabra código transmitida: **0 1 1 0 0 1 1**

$p_1{=}0$ $p_2{=}1$ $d_1{=}1$ $p_3{=}0$ $d_2{=}0$ $d_3{=}1$ $d_4{=}1$

Ejemplo Práctico — Detección y Corrección de Error

Palabra recibida (con error en bit 5): 0 1 1 0 1 1 1 ← bit 5 cambió 0→1

Cálculo del Síndrome:

c_1 (cubre 1,3,5,7)	$0 \oplus 1 \oplus 1 \oplus 1 = 1$	FALLA
c_2 (cubre 2,3,6,7)	$1 \oplus 1 \oplus 1 \oplus 1 = 0$	OK
c_3 (cubre 4,5,6,7)	$0 \oplus 1 \oplus 1 \oplus 1 = 1$	FALLA

Síndrome = $c_3c_2c_1$ = **1 0 1** (binario) = **5** (decimal) → Error en posición 5 → Invertir bit 5

✔ Palabra corregida: **0 1 1 0 0 1 1** → Datos recuperados: **1 0 1 1**

Comparación de Códigos Detectores/Correctores

Código	Bits extra	Detección	Corrección	Overhead	Aplicación típica
Paridad simple	1 bit	1 error	Ninguna	Muy bajo	UART, USB
Hamming (7,4)	3 bits	2 errores	1 error	43%	Memoria RAM ECC
Hamming (15,11)	4 bits	2 errores	1 error	27%	DRAM, almacenamiento
SECDED Hamming	+1 bit	2 errores	1 error	~30%	Servidores ECC
Reed-Solomon	Variable	Ráfagas	Ráfagas	Medio	DVD, QR, espacio
LDPC	Variable	Alta tasa	Alta tasa	Variable	5G, WiFi 6, SSD

Tanenbaum, A.S. & Wetherall, D. (2011). Computer Networks (5th ed.). Prentice Hall.

Aplicaciones del Código de Hamming en la Industria

Memoria RAM ECC

Los módulos ECC (Error Correcting Code) en servidores usan SECDED Hamming. Corrigen errores de 1 bit en tiempo real sin interrumpir el sistema operativo. Esencial en datacenters, aplicable en NET3MAX.

Comunicaciones Espaciales

La NASA y ESA usan Hamming extendido en sondas Voyager, Mars Rovers. El tiempo de propagación hace imposible la retransmisión; la corrección en destino es crítica.

Almacenamiento óptico

CDs, DVDs y Blu-ray usan Reed-Solomon (derivado de Hamming). Los arañazos del disco son errores en ráfaga que el decodificador corrige sin pérdida de datos.

Redes y RAID

RAID-2 usa Hamming entre múltiples discos para corrección de errores distribuida. Ethernet usa CRC (paridad cíclica), pariente de los principios de Hamming.

Chipsets / SoC

Procesadores modernos (ARM Cortex-A, Intel Xeon) incluyen hardware dedicado a Hamming en sus cachés L1/L2 para tolerancia a fallas por radiación cósmica.

FPGAs y Sistemas Embebidos

Xilinx y Intel (Altera) FPGAs implementan Hamming en sus bloques de memoria BRAM para protección en entornos industriales y aeroespaciales.

Implementación en Python — Hamming(7,4)

```python
def hamming_encode(data):  # data: list of 4 bits
    d1,d2,d3,d4 = data
    p1 = d1^d2^d4  # XOR posiciones 1,3,5,7
    p2 = d1^d3^d4  # XOR posiciones 2,3,6,7
    p3 = d2^d3^d4  # XOR posiciones 4,5,6,7
    return [p1,p2,d1,p3,d2,d3,d4]

def hamming_decode(word):  # word: list of 7 bits
    p1,p2,d1,p3,d2,d3,d4 = word
    c1 = p1^d1^d2^d4
    c2 = p2^d1^d3^d4
    c3 = p3^d2^d3^d4
    err_pos = c3*4 + c2*2 + c1  # síndrome
    if err_pos: word[err_pos-1] ^= 1  # corregir
    return word[2],word[4],word[5],word[6]  # datos
```

Distancia de Hamming — Fundamento Matemático

Definición: d(x,y) = número de posiciones en que dos palabras código difieren = peso de Hamming de $x \oplus y$

Ejemplo:

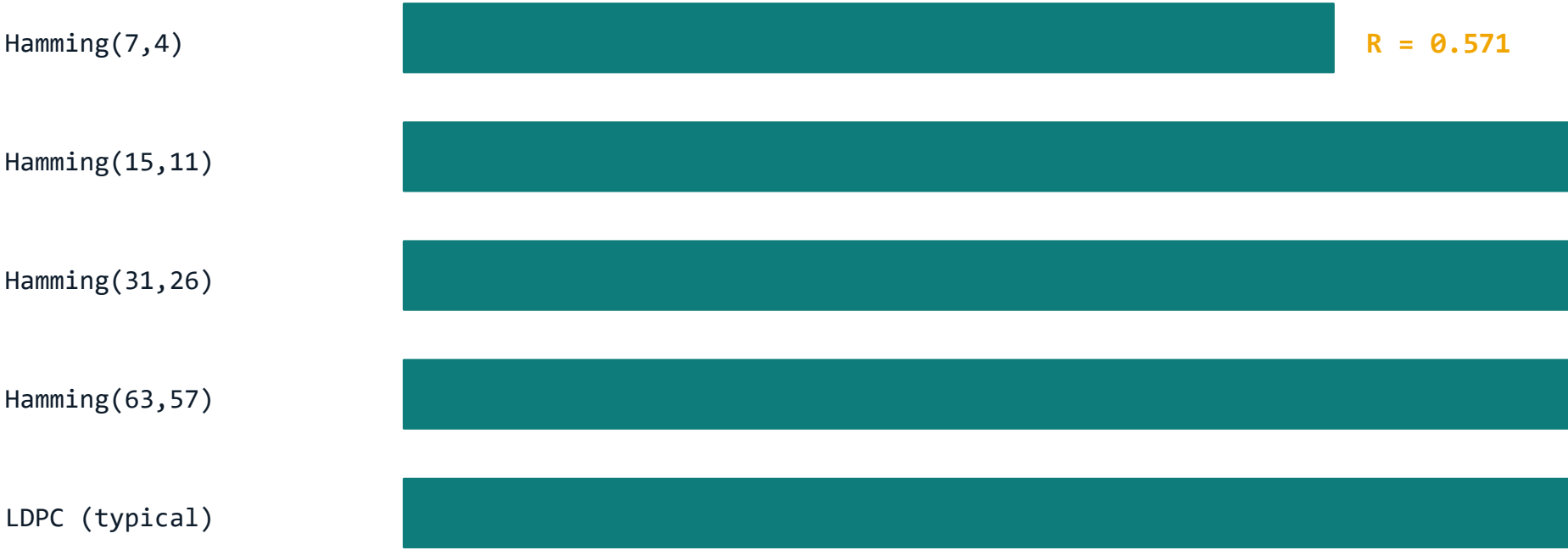
x = 1 0 1 1 0	y = 1 1 1 0 0	diferencias = _ 1 _ 1 _	d = 2
x = 0 1 0 0 1	y = 1 1 0 1 1	diferencias = 1 _ _ 1 _	d = 2
x = 1 0 1 1 0	y = 0 1 0 0 1	diferencias = 1 1 1 1 1	d = 5

Reglas de diseño:

- Distancia mínima d=3 → detectar 2 errores, corregir 1
- Distancia mínima d=4 → detectar 3 errores, corregir 1 (SECDED)
- En general: detectar e errores ↔ $d \geq e+1$ | corregir t errores ↔ $d \geq 2t+1$

Eficiencia y Rendimiento — Análisis Comparativo

Tasa de código R = k/n (bits útiles / bits totales):



A mayor n, mayor eficiencia. Hamming(63,57) es el más común en RAM ECC de servidor de doble socket (2024).

Relevancia en Infraestructura — NET3MAX

¿Por qué importa Hamming en un datacenter moderno?

1

RAM ECC en Servidores HP Z4 G4

Los nodos Xeon W-2123 de NET3MAX soportan ECC UDIMM. Con Hamming SECDED, un bit flip en RAM (por radiación cósmica o ruido eléctrico) se corrige sin downtime — crítico para ISPConfig y MariaDB en producción.

2

Integridad en Storage / RAID

RAID-6 (doble paridad) usa principios derivados de Hamming. Permite que 2 discos fallen simultáneamente sin pérdida de datos. Aplicable en los nodos de almacenamiento de Pods A y B.

3

Transmisión confiable en redes

HAProxy retransmite HTTP/TCP sobre redes con ruido. Las capas físicas (Ethernet GbE/10GbE) usan CRC-32, heredero directo de los principios de Hamming, para detectar errores de trama antes de que lleguen a HAProxy.

4

Tendencia: Kubernetes + ECC

La próxima capa K8s de NET3MAX operará en nodos con ECC obligatorio. Los nodos Xeon de control-plane con ECC garantizan consistencia de etcd, el corazón del estado del cluster.

CÓDIGOS DE DETECCIÓN Y CORRECCIÓN DE ERRORES

Los estándares actuales de comunicaciones inalámbrica y alámbrica utilizan **códigos de detección y corrección de errores** para garantizar la **confiabilidad** e **integridad** de la información transmitida.

COMUNICACIONES INALÁMBRICAS

ESTÁNDAR	CÓDIGOS UTILIZADOS
Wi-Fi (IEEE 802.11)	• CRC (Cyclic Redundancy Check) • LDPC (Low-Density Parity-Check) • Convolucionales (Viterbi)
5G NR (3GPP)	• CRC • LDPC • Polar Codes
Bluetooth (IEEE 802.15.1)	• Hamming Code • CRC
4G LTE (3GPP)	• CRC • Turbo Codes • Convolucionales

COMUNICACIONES ALÁMBRICAS

ESTÁNDAR	CÓDIGOS UTILIZADOS
Ethernet (IEEE 802.3)	• CRC (Frame Check Sequence)
USB (USB 2.0/3.x/4)	• CRC5, CRC16, CRC32 (según el tipo de paquete)
PCI Express (PCIe)	• CRC (32 bits)
HDMI	• TMDS: CRC • FRL: CRC + FEC

Estos códigos permiten **detectar errores** y, en muchos casos, **corregirlos**, asegurando comunicaciones más robustas, eficientes y confiables en todo tipo de redes.

Conclusiones

- El código de Hamming es el pilar de todos los sistemas modernos de corrección de errores en hardware.
- Los bits de paridad en potencias de 2, el cálculo XOR y el síndrome permiten corregir 1 bit en tiempo O(log n).
- La distancia mínima d=3 garantiza SECDED: 1 corrección + 2 detección, fundamento de RAM ECC.
- Evolucionó hacia Reed-Solomon, LDPC y Turbo Codes, base de 5G, almacenamiento SSD y comunicación espacial.
- En datacenters como NET3MAX, Hamming se aplica directamente en hardware ECC y RAID de producción.

Referencias Bibliográficas

- Hamming, R. W. (1950). Error detecting and error correcting codes. Bell System Technical Journal, 29(2), 147–160. https://doi.org/10.1002/j.1538-7305.1950.tb00463.x
- Lin, S., & Costello, D. J. (2004). Error Control Coding: Fundamentals and Applications (2nd ed.). Pearson Prentice Hall.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Prentice Hall. ISBN: 978-0132126953.
- Moon, T. K. (2005). Error Correction Coding: Mathematical Methods and Algorithms. Wiley-Interscience.
- Forouzan, B. A. (2012). Data Communications and Networking (5th ed.). McGraw-Hill. ISBN: 978-0073376226.
- Intel Corporation. (2023). Intel Xeon Scalable Processors – ECC Memory Support. Technical Brief. https://www.intel.com
- JEDEC Solid State Technology Association. (2020). JEDEC Standard JESD79-5B: DDR5 SDRAM. JEDEC.
- Sklar, B. (2001). Digital Communications: Fundamentals and Applications (2nd ed.). Prentice Hall PTR.

Todas las referencias siguen el formato APA 7.ª edición.

PROTOCOLOS DE CONTROL DE FLUJO ARQ (AUTOMATIC REPEAT REQUEST)

STOP-AND-WAIT (S&W)

Transmite 1 paquete y espera **confirmación** (ACK)

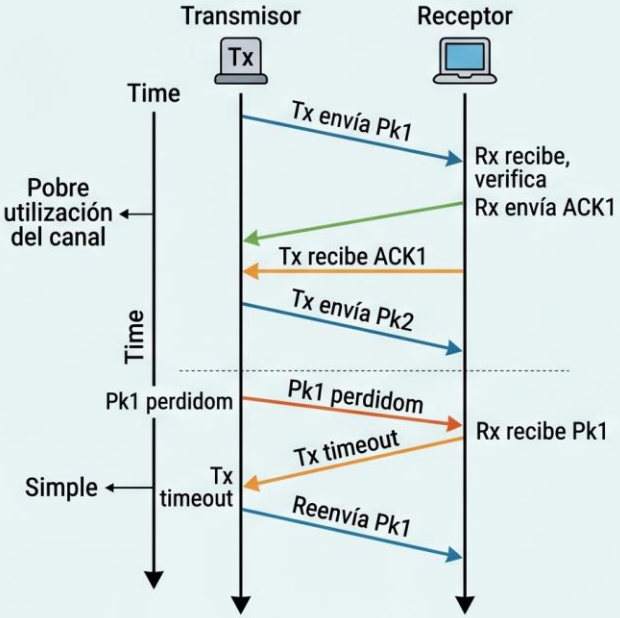
GO-BACK-N (GBN)

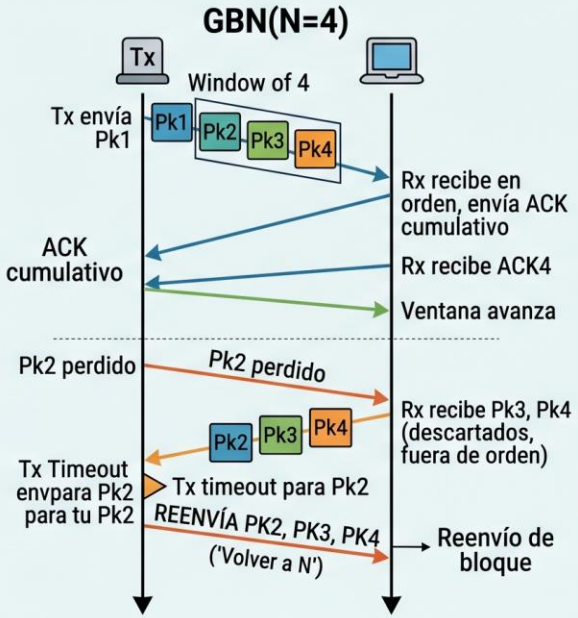
espera

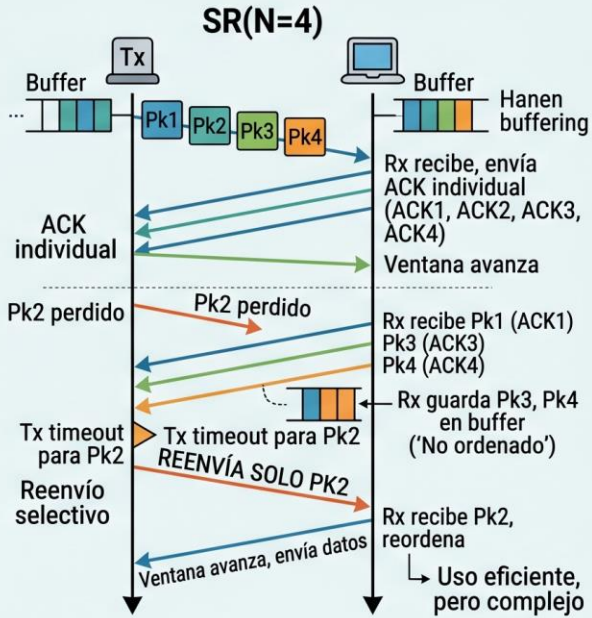
Transmite N paquetes y solo **confirmación** (ACK) del último

SELECTIVE REPEAT (SR)

Transmite N paquetes y espera confirmación individual de cada uno







COMPARACIÓN BÁSICA

- Eficiencia baja
- Uso simple

- Eficiencia media
- Reenvío masivo

- Eficiencia alta
- Uso complejo (memoria)